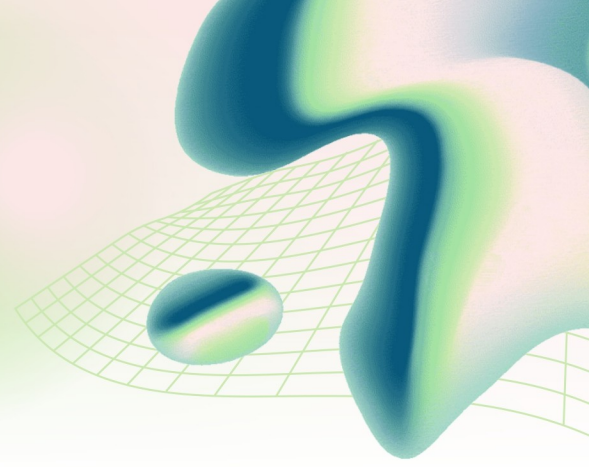


HABITLY

Sistema de Gestión Inmobiliaria en Java



Anexo 02 Estrategia de Privacidad y Encriptación AES

Versión 1.0.1A

Documento de evolución funcional y técnica

Autor: DevNaranjo

Año: 2026

Índice

1. Introducción.....	4
2. Contexto Actual de Persistencia.....	5
2.1 Flujo actual.....	5
2.2 Ventajas de la solución actual.....	5
3. Problema Principal: La Serialización No Es Seguridad.....	6
3.1 En otras palabras:.....	6
4. Riesgos de Seguridad del Sistema Actual.....	7
4.1 Lectura no autorizada de datos.....	7
4.2 Exposición futura de información personal.....	7
4.3 Riesgo de copia, duplicado o filtración.....	8
4.4 Falta de preparación para una evolución real del sistema.....	8
5. Objetivo del Anexo.....	9
6. Principio de Seguridad Aplicado.....	10
6.1 Situación deseada.....	10
7. Solución Propuesta: Cifrado Simétrico AES-128.....	11
8. ¿Qué es AES?.....	12
8.1 Características principales.....	12
9. ¿Por qué AES-128 y no otra opción?.....	13
9.1 Razones técnicas.....	13
10. Flujo de Seguridad Recomendado.....	14
10.1 Fase de guardado.....	14
10.2 Fase de carga.....	14
11. Beneficio Técnico de este Enfoque.....	15
11.1 El modelo de datos no necesita cambiar.....	15
12. Vulnerabilidad Actual del Archivo Serializado.....	16
12.1 Riesgo conceptual importante.....	16
13. Protección de Datos de Inquilinos y Usuarios.....	17
14. Diseño de la Clave de Cifrado.....	18
15. Qué NO conviene hacer.....	19
16. Qué SÍ conviene hacer.....	20
17. Consideración Importante: Seguridad Realista para Habitly.....	21
17.1 Esta mejora no convierte Habitly en un sistema blindado absoluto.....	21
18. Riesgos y Límites de la Solución.....	22
18.1 Limitaciones principales.....	22
19. Requisitos Funcionales Asociados.....	23
19.1 RF-II.1.....	23
19.2 RF-II.2.....	23
19.3 RF-II.3.....	23
19.4 RF-II.4.....	23
19.5 RF-II.5.....	23
20. Requisitos No Funcionales Asociados.....	24
20.1 RNF-II.1 — Privacidad.....	24

Anexo 02 – v1.0.1A

20.2 RNF-II.2 — Robustez.....	24
20.3 RNF-II.3 — Mantenibilidad.....	24
20.4 RNF-II.4 — Escalabilidad.....	24
20.5 RNF-II.5 — Transparencia de uso.....	24
21. Integración Recomendada en la Arquitectura de Habitly.....	25
21.1 Arquitectura lógica recomendada.....	25
22. Relación con el Anexo 01.....	26
22.1 Anexo 01 aporta:.....	26
22.2 Anexo 02 aporta:.....	26
23. Riesgos si No se Implementa esta Mejora.....	27
24. Conclusión.....	28

1. INTRODUCCIÓN

En la versión **v1.0.1-BETA**, Habitly incorporó un sistema funcional de persistencia basado en **serialización de objetos Java**, permitiendo guardar el inventario de viviendas y su estado financiero en un archivo binario .dat.

Esta solución ha sido suficiente para la fase inicial del proyecto, ya que permite:

- conservar los datos entre ejecuciones,
- evitar pérdida de información al cerrar el programa,
- y sentar una base técnica para futuras mejoras.

Sin embargo, desde una perspectiva de **seguridad y privacidad**, esta implementación sigue siendo insuficiente para un sistema que, en próximas versiones, gestionará datos más delicados relacionados con:

- inquilinos,
- propietarios,
- historial de pagos,
- posibles gastos recurrentes,
- trazabilidad económica,
- y futura identificación de usuarios.

Por ello, antes de avanzar hacia una versión más completa del sistema, se considera necesario definir una **estrategia de protección de datos en reposo**, incorporando un sistema de **cifrado simétrico AES-128** antes de escribir la información al disco.

Este anexo establece la justificación técnica, los riesgos del sistema actual y la estrategia recomendada para su evolución.

2. CONTEXTO ACTUAL DE PERSISTENCIA

Actualmente, Habitly guarda la información principal del sistema mediante un proceso de:

2.1 Flujo actual

- el `ArrayList<Vivienda>` se serializa,
- se transforma en un flujo binario,
- y se almacena directamente en un archivo:

`inventario_habitly.dat`

Esto significa que el programa puede reconstruir posteriormente la estructura completa de objetos al volver a ejecutarse.

2.2 Ventajas de la solución actual

- simplicidad de implementación,
- integración natural con Java,
- rapidez para una fase beta,
- y persistencia funcional sin necesidad de base de datos.

No obstante, esta estrategia **no protege realmente la información**.

3. PROBLEMA PRINCIPAL: LA SERIALIZACIÓN NO ES SEGURIDAD

Un error común en proyectos en crecimiento es asumir que, por el hecho de guardar datos en un archivo .dat binario, estos ya están “ocultos” o “protegidos”.

Esto es **incorrecto**.

La serialización de objetos Java **no cifra la información**. Solo la transforma a un formato binario que el sistema puede reconstruir más tarde.

3.1 En otras palabras:

Un archivo serializado puede ser ilegible a simple vista para un usuario básico, pero sigue siendo técnicamente accesible y vulnerable.

Por tanto, **serializar no equivale a securizar**.

4. RIESGOS DE SEGURIDAD DEL SISTEMA ACTUAL

Si Habitly sigue guardando los datos serializados directamente en disco sin ninguna capa de cifrado, se mantienen varios riesgos relevantes.

4.1 Lectura no autorizada de datos

Una persona con acceso al equipo podría:

- copiar el archivo .dat,
- moverlo a otro entorno Java,
- intentar deserializarlo,
- o inspeccionarlo con herramientas técnicas.

Aunque no sea una lectura “bonita” o inmediata, **la información no está protegida de verdad.**

4.2 Exposición futura de información personal

Actualmente el sistema gira principalmente en torno a viviendas y estados de pago, pero el roadmap de Habitly contempla incorporar más adelante datos como:

- nombre de arrendatarios,
- historial de pagos,
- deudas,
- información económica,
- gastos asociados,
- posibles perfiles de usuario,
- y trazabilidad de operaciones.

Esto convierte al archivo persistente en un **activo sensible**.

Guardar esa información sin cifrado sería una mala práctica.

4.3 Riesgo de copia, duplicado o filtración

Un archivo .dat no cifrado puede:

- copiarse fácilmente,
- moverse entre equipos,
- o quedar expuesto en backups inseguros.

Esto compromete la privacidad de los datos gestionados por Habitly.

4.4 Falta de preparación para una evolución real del sistema

Aunque Habitly aún no sea una aplicación comercial desplegada, **ya está creciendo con lógica de negocio real.**

Por tanto, seguir usando persistencia sin cifrado más allá de la beta generaría deuda técnica y un mal punto de partida para la v1.1.

5. OBJETIVO DEL ANEXO

El objetivo de este anexo es definir una estrategia técnica para que Habitly:

- proteja los datos persistentes en disco,
- dificulte accesos no autorizados,
- refuerce la privacidad del usuario,
- y prepare una base sólida de seguridad para futuras ampliaciones.

La medida propuesta consiste en incorporar un proceso de:

Cifrado AES-128 antes de guardar en disco

y su correspondiente:

Descifrado AES-128 antes de cargar en memoria

6. PRINCIPIO DE SEGURIDAD APLICADO

La mejora propuesta se apoya en un principio esencial de seguridad informática:

Los datos sensibles deben almacenarse cifrados cuando están en reposo.

Esto significa que, aunque el programa no esté ejecutándose, la información almacenada físicamente en el disco debe seguir protegida.

En Habitly esto afecta directamente al archivo de inventario persistente.

6.1 Situación deseada

- En memoria: los objetos se manejan normalmente.
- En disco: los datos deben quedar cifrados.

De esta forma, si alguien accede directamente al archivo físico, **no podrá interpretarlo sin la clave correspondiente.**

7. SOLUCIÓN PROPUESTA: CIFRADO SIMÉTRICO AES-128

La estrategia recomendada para Habitly en esta fase es el uso de:

AES-128

8. ¿QUÉ ES AES?

AES (**Advanced Encryption Standard**) es uno de los algoritmos de cifrado simétrico más utilizados y aceptados en la industria.

8.1 Características principales

- estándar ampliamente reconocido,
- rápido y eficiente,
- adecuado para archivos locales,
- fácil de integrar con Java,
- suficientemente robusto para el contexto actual de Habitly.

¿Qué significa “simétrico”?

Significa que:

- se usa **la misma clave** para cifrar
- y para descifrar.

Esto lo hace especialmente práctico para una aplicación local de escritorio o consola como Habitly.

9. ¿POR QUÉ AES-128 Y NO OTRA OPCIÓN?

Para el momento actual del proyecto, **AES-128** es una elección equilibrada.

9.1 Razones técnicas

- Es suficientemente seguro para la escala actual del proyecto.
- Tiene una integración sencilla en Java.
- Evita complejidad innecesaria frente a soluciones más avanzadas.
- Es una mejora realista y asumible antes de la v1.1.

Importante

No se busca ahora mismo construir un sistema criptográfico complejo, sino:

- **subir el nivel de seguridad real**
- sin sobreingeniería
- y manteniendo el proyecto entendible.

Esto encaja muy bien con la filosofía actual de Habitly.

10. FLUJO DE SEGURIDAD RECOMENDADO

Habitly no debería escribir directamente el objeto serializado al archivo final.

En su lugar, el flujo recomendado sería el siguiente:

10.1 Fase de guardado

1. El sistema reúne en memoria el inventario actual.
2. Ese inventario se serializa temporalmente.
3. El flujo binario resultante se cifra mediante AES-128.
4. Solo el contenido ya cifrado se escribe en disco.

10.2 Fase de carga

1. Habitly localiza el archivo persistente.
2. Lee el contenido cifrado desde disco.
3. Descifra el contenido con la clave correcta.
4. Deserializa el contenido ya recuperado.
5. Reconstruye el `ArrayList<Vivienda>` en memoria.

11. BENEFICIO TÉCNICO DE ESTE ENFOQUE

Este diseño tiene una ventaja muy importante:

11.1 El modelo de datos no necesita cambiar

Es decir:

- Vivienda, Piso, Casa y demás clases pueden seguir funcionando igual.
- La seguridad se aplica **en la capa de persistencia**, no en la capa de negocio.

Esto es muy positivo porque permite:

- mantener el código limpio,
- no romper la lógica funcional,
- y encapsular la mejora donde realmente corresponde.

12. VULNERABILIDAD ACTUAL DEL ARCHIVO SERIALIZADO

Aunque el archivo .dat no sea un documento de texto plano, su contenido sigue siendo vulnerable porque:

- no está protegido criptográficamente,
- puede ser copiado o reemplazado,
- y depende únicamente de “estar escondido” o “ser raro”.

Eso no es seguridad real.

12.1 Riesgo conceptual importante

“Que un archivo no sea cómodo de leer no significa que esté protegido”.

Por eso la seguridad no puede basarse en:

- la invisibilidad,
- la ubicación,
- ni el desconocimiento del usuario.

Debe basarse en **protección técnica efectiva**.

13. PROTECCIÓN DE DATOS DE INQUILINOS Y USUARIOS

Aunque en la versión actual Habitly aún no haya desplegado un sistema completo de usuarios, esta mejora se plantea con visión de continuidad.

A medio plazo, Habitly podría almacenar datos como:

- identificadores de arrendatarios,
- historial de alquileres,
- pagos parciales o completos,
- gastos por vivienda,
- registros económicos,
- e incluso trazabilidad de acciones.

Todo esto constituye información con valor funcional y cierta sensibilidad.

Por ello, esta capa de cifrado debe entenderse como una **inversión estructural temprana** y no como una mejora opcional.

14. DISEÑO DE LA CLAVE DE CIFRADO

Uno de los puntos más importantes de esta estrategia no es solo cifrar, sino **cómo gestionar la clave.**

Problema

AES-128 requiere una clave.

Y si esa clave se guarda mal, la seguridad pierde valor.

15. QUÉ NO CONVIENE HACER

Habitly **no debería**:

- dejar la clave escrita directamente en múltiples clases,
- repetirla por el proyecto,
- ponerla en comentarios,
- o dejarla dispersa dentro del código sin control.

Eso generaría una mala práctica de seguridad y dificultaría el mantenimiento.

16. QUÉ SÍ CONVIENE HACER

Para esta fase, la estrategia más razonable es:

- centralizar la clave en una única capa de seguridad,
- encapsular la lógica de cifrado y descifrado,
- y dejar el sistema preparado para una mejora posterior.

Recomendación técnica

Crear más adelante una clase responsable del cifrado, por ejemplo:

`GestorSeguridad`

o

`CifradorAES`

Posibles responsabilidades

- cifrar datos antes de guardar
- descifrar datos al cargar
- centralizar la clave
- abstraer la complejidad criptográfica del resto del sistema

Esto evita contaminar la lógica de negocio con detalles de seguridad.

17. CONSIDERACIÓN IMPORTANTE: SEGURIDAD REALISTA PARA HABITLY

Conviene dejar claro algo importante:

17.1 Esta mejora no convierte Habitly en un sistema blindado absoluto

Pero sí lo convierte en un sistema **mucho más serio y responsable**.

Eso es exactamente lo que interesa ahora.

La meta de esta fase no es construir una plataforma bancaria, sino:

- eliminar vulnerabilidades básicas,
- elevar la madurez del proyecto,
- y dejar la base preparada para futuras versiones.

Ese enfoque es técnicamente sano.

18. RIESGOS Y LÍMITES DE LA SOLUCIÓN

Como toda mejora, esta estrategia también tiene limitaciones que conviene documentar.

18.1 Limitaciones principales

- Si la clave está embebida localmente, sigue existiendo un nivel de exposición técnica.
- El cifrado protege el archivo en reposo, pero no sustituye otras capas de seguridad.
- No protege frente a todo tipo de manipulación avanzada del sistema.

Aun así, para el contexto actual de Habitly, el beneficio es muy superior al coste de implementación.

19. REQUISITOS FUNCIONALES ASOCIADOS

Para considerar correctamente implementada esta mejora, Habitly deberá cumplir los siguientes requisitos:

19.1 RF-II.1

El sistema deberá cifrar el inventario antes de escribirlo en disco.

19.2 RF-II.2

El sistema deberá descifrar correctamente el archivo antes de reconstruir los objetos.

19.3 RF-II.3

La persistencia cifrada deberá mantener la compatibilidad funcional con la lógica actual del inventario.

19.4 RF-II.4

El usuario no deberá percibir una complejidad adicional durante el uso normal del sistema.

19.5 RF-II.5

La capa de seguridad deberá estar desacoplada de la lógica principal de negocio.

20. REQUISITOS NO FUNCIONALES ASOCIADOS

20.1 RNF-II.1 — Privacidad

Los datos persistentes no deberán quedar accesibles en bruto sobre el disco.

20.2 RNF-II.2 — Robustez

La aplicación deberá gestionar correctamente errores de lectura o descifrado.

20.3 RNF-II.3 — Mantenibilidad

La lógica criptográfica deberá centralizarse y no repartirse por múltiples clases.

20.4 RNF-II.4 — Escalabilidad

La solución deberá ser compatible con futuras ampliaciones del modelo de datos.

20.5 RNF-II.5 — Transparencia de uso

El usuario no deberá tener que intervenir manualmente para cifrar o descifrar.

21. INTEGRACIÓN RECOMENDADA EN LA ARQUITECTURA DE HABITLY

Desde una perspectiva de diseño, esta mejora debería integrarse dentro del bloque de persistencia del sistema.

21.1 Arquitectura lógica recomendada

- GestionInmobiliaria
 - controla el flujo del programa
- GestorPersistencia
 - gestiona guardado y carga
- CifradorAES
 - cifra y descifra el contenido
- Vivienda / Piso / Casa
 - mantienen la lógica de negocio y el modelo de datos

Esto permite una separación mucho más limpia entre:

- negocio,
- almacenamiento,
- y seguridad.

22. RELACIÓN CON EL ANEXO 01

Este anexo está estrechamente relacionado con el **Anexo I — Protocolo de Rutas de Sistema y Almacenamiento Oculto**.

La combinación de ambas mejoras proporciona una protección mucho más sólida:

22.1 Anexo 01 aporta:

- mejor ubicación del archivo
- menor exposición operativa
- mejor desacoplamiento del proyecto

22.2 Anexo 02 aporta:

- protección real del contenido del archivo
- privacidad técnica de los datos
- endurecimiento de la persistencia

Conclusión técnica conjunta

Guardar el archivo en una ruta oculta ayuda, pero cifrarlo es lo que realmente protege su contenido.

Ambas mejoras deben entenderse como complementarias.

23. RIESGOS SI NO SE IMPLEMENTA ESTA MEJORA

Si Habitly continúa almacenando el inventario serializado sin cifrado, se mantendrán los siguientes problemas:

- exposición innecesaria de datos locales,
- mala práctica de seguridad,
- mayor fragilidad de cara a la evolución del sistema,
- y acumulación de deuda técnica antes de la v1.1.

A medida que Habitly incorpore más información económica y operativa, esta vulnerabilidad dejará de ser tolerable.

24. CONCLUSIÓN

La implementación de un sistema de cifrado simétrico **AES-128** para la persistencia local de Habitly representa una mejora crítica en términos de **privacidad, madurez técnica y responsabilidad estructural**.

Aunque la serialización de objetos Java ha permitido construir una base funcional rápida y eficaz durante la fase beta, su uso sin protección criptográfica resulta insuficiente para una aplicación que ya empieza a gestionar datos con valor operativo real.

Adoptar un modelo en el que el inventario se **serialice, se cifre y solo entonces se escriba en disco** permitirá a Habitly:

- proteger mejor la información,
- reducir la exposición local de datos,
- mantener una arquitectura limpia,
- y preparar el proyecto para futuras ampliaciones funcionales.

Este anexo debe considerarse una guía de implementación prioritaria dentro del endurecimiento técnico previsto para la evolución de Habitly hacia la línea **v1.1**.